UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CÂMPUS PONTA GROSSA
DEPARTAMENTO DE COMPUTAÇÃO

Francis Henrique Batista Cunha
Kemuel Lucas Vieira

RELATÓRIO TÉCNICO: CONSTRUÇÃO DE ANALISADORES LÉXICO E SINTÁTICO

Disciplina de Compiladores - 2026.1

Ponta Grossa
2026

Sumário

1 Introdução **2**

2 Análise Léxica **2**

2.1 Identificação de Tokens e Lexemas 2

2.2 Diagramas de Transição (Debuggex) 2

3 Análise Sintática **3**

3.1 Gramática em Notação EBNF 3

3.2 Diagramas de Sintaxe (Railroad Diagrams) 4

4 Detalhamento da Implementação **6**

4.1 Especificação Flex 7

4.2 Especificação Bison 7

5 Uso de Ferramentas de IA **8**

6 Conclusão **8**

7 Referências Bibliográficas **9**

1 Introdução

Esse documento descreve o complemento e a implementação de uma linguagem básica de programação, baseando-se na calculadora vista em sala de aula.

2 Análise Léxica

2.1 Identificação de Tokens e Lexemas

Para complementar a análise léxica, foi adicionado o reconhecimento para os tokens "for", "and"e "or", que eram os elementos que deveriam ser implementados na linguagem.

2.2 Diagramas de Transição (Debuggex)

Para representar as regras léxicas, foi utilizado a ferramenta Debuggex. As seguintes regras foram representadas:

```
NAME      [a-zA-Z][a-zA-Z0-9]*
NUMBER    [0-9]+"."[0-9]*{EXP}? | "."?[0-9]+{EXP}?
EXP       ([Ee][-+]?[0-9]+)
```

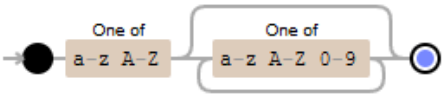


Figura 1: Diagrama de Transição para Identificadores (NAME).

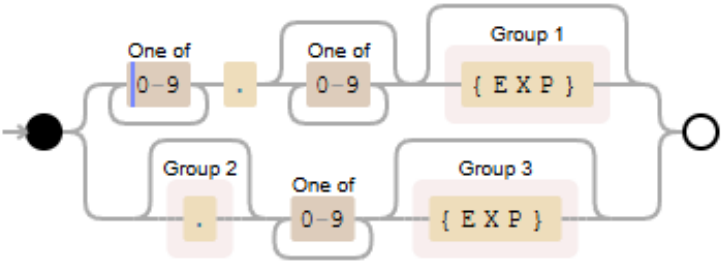


Figura 2: Diagrama de Transição para Identificadores (NUMBER).

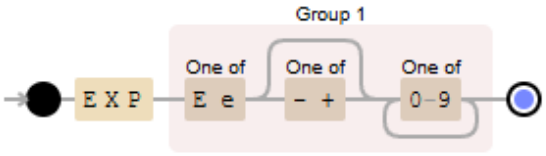


Figura 3: Diagrama de Transição para Identificadores (EXP).

3 Análise Sintática

3.1 Gramática em Notação EBNF

Apresentação da gramática expandida utilizando a notação EBNF.

```
calclist ::= (calclist ( stmt EOL
                       | LET NAME '(' symlist ')' '=' list EOL
                       | 'error' EOL ))*

stmt ::= 'IF' exp 'THEN' list ( 'ELSE' list )?
       | 'WHILE' exp 'DO' list
       | 'FOR' '(' exp ';' exp ';' exp ')' list
       | exp

list ::= ( stmt ';' )*

exp ::= exp ( CMP | '+' | '-' | 'AND' | 'OR' ) exp
      | NAME '=' exp
      | '(' exp ')'
      | NUMBER
      | NAME
      | FUNC '(' explist ')'
      | NAME '(' explist ')'

explist ::= exp ( ',' exp )*

symlist ::= NAME ( ',' NAME )*
```

3.2 Diagramas de Sintaxe (Railroad Diagrams)

Visualização da estrutura gramatical através da ferramenta Bottlecaps (Railroad Diagram).

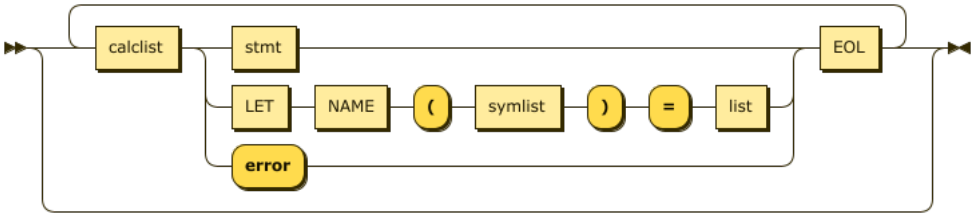


Figura 4: Diagrama de Sintaxe (calclist).

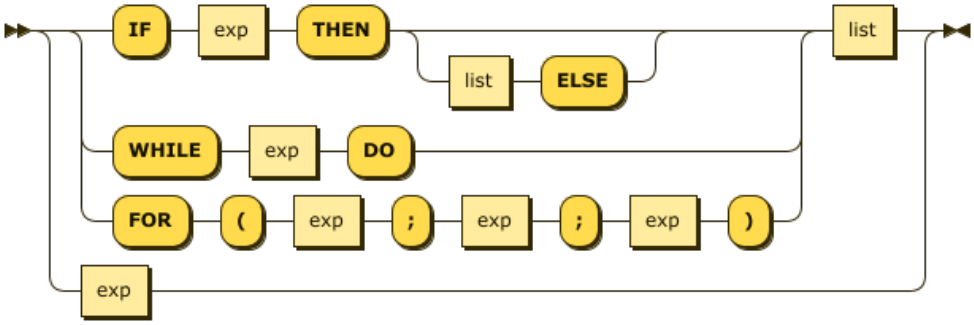


Figura 5: Diagrama de Sintaxe (stmt).

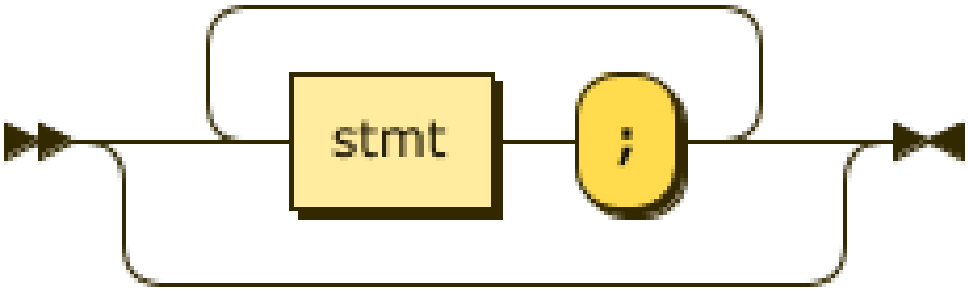


Figura 6: Diagrama de Sintaxe (list).

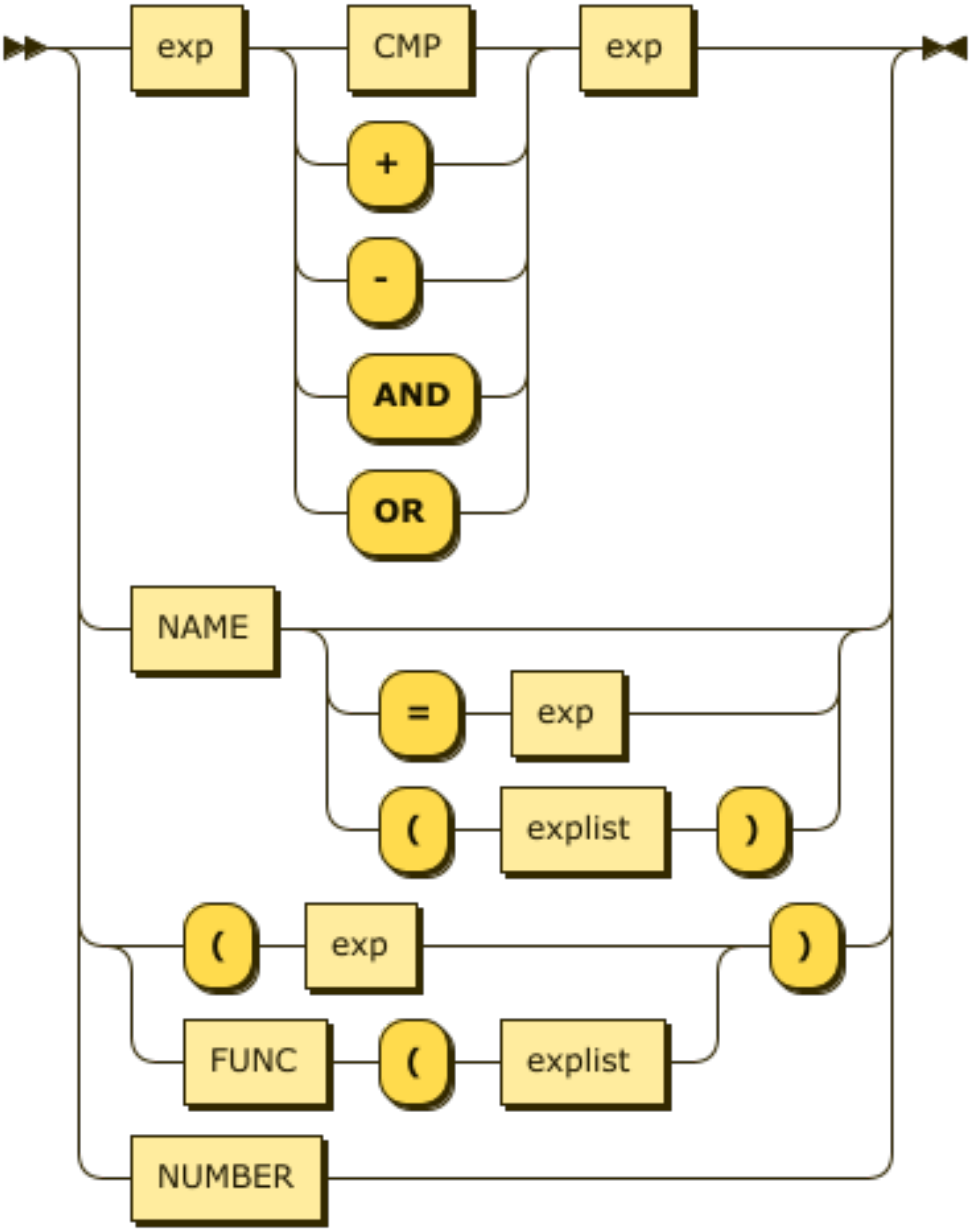


Figura 7: Diagrama de Sintaxe (exp).

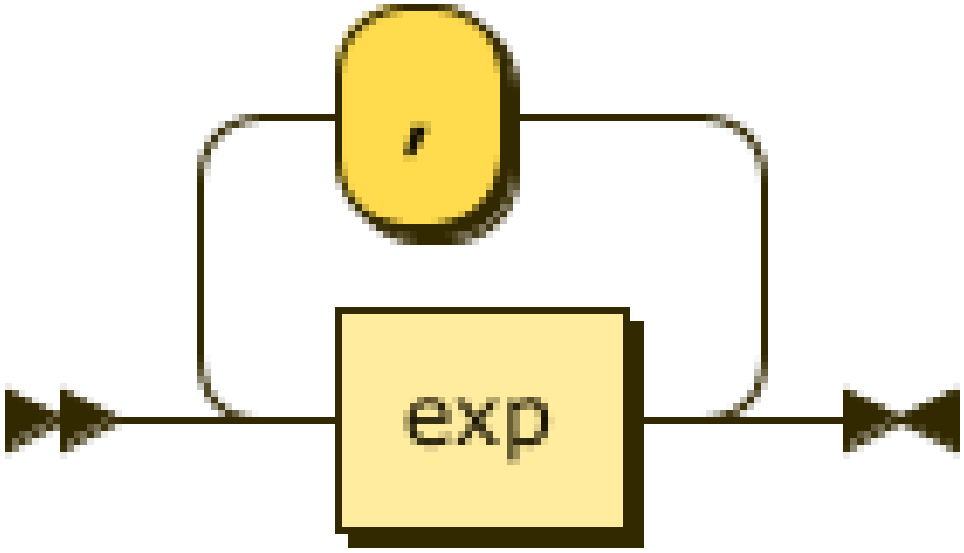


Figura 8: Diagrama de Sintaxe (explist).

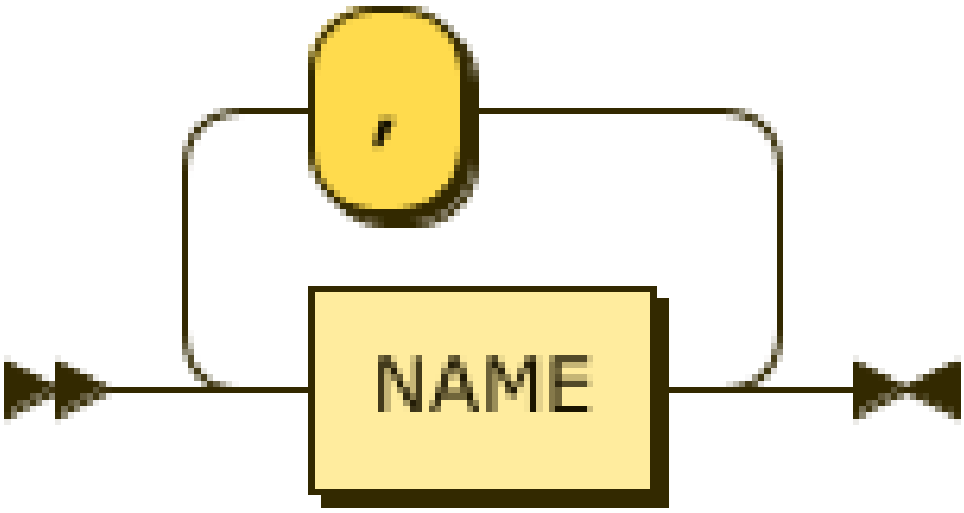


Figura 9: Diagrama de Sintaxe (symlist).

4 Detalhamento da Implementação

Nos arquivos de cabeçalho e de implementação das funções é que ocorreram as maiores modificações. Começando pelos arquivos de cabeçalho, em que optou-se pela criação de uma nova struct, que possui uma ast para cada um dos três parâmetros, além de uma ast para a lista de comandos a serem executados. Escolheu-se utilizar um tipo definido para o nodetype para facilitar a implementação da função “eval”.

```
1 struct _for {
2     int nodetype;     /* tipo NODETYPE_FOR */
3     struct ast *init; /* variavel de inicializacao */
```

```
    struct ast *cond; /* condicao de parada */
    struct ast *inc;  /* incremento */
    struct ast *tl;   /* lista de comandos do corpo */
};
```

Listing 1: Struct para regra for

Para as funcionalidades de “and” e “or” optou-se por reutilizar a lógica das regras de comparação e aproveitar o uso da struct ast padrão assim como para outros operadores.

No arquivo .c foi implementado a função “newfor” seguindo a lógica para as demais funções já existentes. Outras duas funções em que foram necessários ajustes, foram as funções “treefree” e a anteriormente citada, “eval”.

```
case NODETYPE_FOR:
    v = 0.0;
    if (((struct _for *) a)->tl)
        for (eval(((struct _for *) a)->init);
            eval(((struct _for *) a)->cond);
            eval(((struct _for *) a)->inc))
            v = eval(((struct _for *) a)->tl);
    break;
```

Listing 2: Switch case para a regra for na função eval

```
case NODETYPE_FOR:
    if(((struct _for *)a)->init) treefree(((struct _for *) a)->init);
    if(((struct _for *)a)->cond) treefree(((struct _for *) a)->cond);
    if(((struct _for *)a)->inc)  treefree(((struct _for *) a)->inc);
    if(((struct _for *)a)->tl)   treefree(((struct _for *) a)->tl);
    break;
```

Listing 3: Switch case para a regra for na função treefree

4.1 Especificação Flex

```
"for"    { return FOR; }
"and"    { return AND; }
"or"     { return OR; }
```

Listing 4: Palavras chave adicionadas no analisador léxico

4.2 Especificação Bison

```
stmt : IF exp THEN list           { $$ = newflow('I', $2, $4, NULL);
   }
     | IF exp THEN list ELSE list { $$ = newflow('I', $2, $4, $6); }
     | WHILE exp DO list          { $$ = newflow('W', $2, $4, NULL);
        }
     | FOR '(' exp ';' exp ';' exp ')' list  { $$ = newfor(
        NODETYPE_FOR, $3, $5, $7, $9); }
     | exp
     ;
```

Listing 5: Modificação da regra stmt do parser para identificação do for

```
exp : exp CMP exp             { $$ = newcmp($2, $1, $3); }
    | exp '+' exp             { $$ = newast('+', $1, $3); }
    | exp '-' exp             { $$ = newast('-', $1, $3); }
    | exp '*' exp             { $$ = newast('*', $1, $3); }
    | exp '/' exp             { $$ = newast('/', $1, $3); }
    | exp AND exp             { $$ = newast('&', $1, $3); }
    | exp OR exp              { $$ = newast('|', $1, $3); }
    | NAME '=' exp            { $$ = newasgn($1, $3); }
    | '(' exp ')'             { $$ = $2; }
    | NUMBER                  { $$ = newnum($1); }
    | NAME                    { $$ = newref($1); }
    | FUNC '(' explist ')'    { $$ = newfunc($1, $3); }
    | NAME '(' explist ')'    { $$ = newcall($1, $3); }
    ;
```

Listing 6: Modificação da regra exp para reconhecimendo dos operadores lógicos

5 Uso de Ferramentas de IA

Utilizou-se do uso de ferramentas de IA (ex: Gemini) para o esclarecimento de pequenas dúvidas, principalmente em relação à sintaxe e entendimento do funcionamento comum de linguagens de programação populares, para uso como refêrencia na implementação da linguagem apresentada neste trabalho, além do uso para avaliação do trabalho final e geração de testes adicionais.

6 Conclusão

Utilizando-se de testes prontos foi possível visualizar a robustez da linguagem quanto à diversas situações e diferentes usos da lógica de programação utilizada na implementação do lexer e parser da linguagem.

7 Referências Bibliográficas

ALVES, Gleifer Vaz. **Trabalho Prático 1: Construção de Analisadores Léxico & Sintático**. UTFPR, 2026.

LEVINE, John. **flex & bison**: Text Processing Tools. O'Reilly Media, 2009.